

THE SORRY SCHEME OF THINGS ENTIRE

DISSATISFACTION WITH THE STATE OF THE ART.

THURSDAY, MARCH 20, 2014

Custom tick labels in R perspective plots

In R, the `persp()` is a built-in function to create surface plots. The basic usage is straightforward: create a matrix of values

```
# plot a 10x10 matrix of random values in the range -100..100:
persp( matrix(runif(100, min=-100, max=100), nrow=10, ncol=10) )
```

All well and good, until it's time to prepare the plots for presentation -- and suddenly it becomes apparent that plots created with `persp()` do not work well with `axis()`, `text()`, `mtext()`, `par()`, and other standard graphics device functions.

The `trans3d` documentation refers the reader to the `persp` documentation for examples; those examples are too convoluted to serve any useful educational purpose. A quick note to documentation writers: always include an example showing the simplest possible use of your function on trivial data sets (usually the array of integers from 1 to 10, or `sin(x)` if a function is required). Do not use only edge cases and exciting demos as examples.

The discussion that follows will demonstrate how to construct a perspective plot with custom labels using `persp()` and `trans3d()`. The data to be plotted is a 10x10 matrix of values in the range `-100:100`.

The first thing to understand is that `persp()` does not just draw a plot; it also returns a *perspective matrix* (or `pmat`) which can be used to translate 3-dimensional coordinates to the 2-dimensional coordinate system used in the image of the plot.

The function that performs this translation is `trans3d()`. Its arguments are the **x**, **y**, and **z** coordinates to be translated, followed by the `pmat`. The return value is a list with two elements: `x` and `y`, the two-dimensional coordinates in the image.

If one of the **x**, **y**, and **z** arguments is a vector, then the vector is considered to be a line at the other two coordinates. Thus, a line along the X axis from (0, 10, 10) to (10, 10, 10) would be translated using `trans3d(0:10, 10, 10, pmat)`; a line along the Y axis

LABELS

linux (49)
ruby (30)
ubuntu (28)
r (13)
r500 (11)
e17 (9)
upgrade rage (9)
shell (8)
firefox (7)
qt (7)
programming (6)
git (5)
vim (5)
open source (4)
words (4)
javascript (3)
kde (3)
kernel (3)
binary analysis (2)
book (2)
c/c++ (2)
data science (2)
disasm (2)
finance (2)
gcc (2)
jruby (2)
osx (2)
software engineering (2)
tcp/ip (2)
text analysis (2)
x1 (2)
Google Go (1)
blowfish (1)
database (1)
doxygen (1)
e70 (1)
freebsd (1)

from (0, 3, 10) to (0, 7, 10) would be translated using `trans3d(0, 3:7, 10, pmat)`.

Enough background; time for an example.

Basic Perspective Plot

First, some definitions of the data ranges to keep things clear:

```
x.axis <- 1:10
min.x <- 1
max.x <- 10
y.axis <- 1:10
min.y <- 1
max.y <- 10
z.axis <- seq(-100, 100, by=25)
min.z <- -100
max.z <- 100
```

Pay particular attention to `z.axis`: in addition to specifying the *range* of each axis, the `*.axis` variables also specify the *tick marks* of each axis.

Next, a draw the initial perspective plot, saving the `pmat`:

```
pmat <- persp( x=x.axis, y=y.axis,
               matrix(runif(100, min=-100, max=100), nrow=10,
                        ncol=10),
               xlab='', ylab='', zlab='',
               ticktype='detailed', box=FALSE, axes=FALSE,
               mar=c(10, 1, 0, 2), expand=0.25,
               col='green', shade=0.25, theta=40, phi=30 )
```

Note the `theta` (rotation along the vertical axis) and `phi` (rotation along the horizontal axis) parameters. It is useful to play with these a bit, as different data sets will require different viewing angles. The `r` ("eyepoint distance") and `d` ("perspective strength") parameters provide further control of the view. Note also that `box` and `axes` parameters are `FALSE`: we will be drawing our own axes.

Drawing the Axes

In this plot, the X axis will be drawn at `min.y` and `min.z` (left side of Y, bottom of Z), Y at `max.x` and `min.z` (right side of X, bottom of Z), and Z at `min.x` and `min.y` (left side of X, left side of Y).

These parameters are passed to `trans3d()` to calculate the

grub2 (1)
 macports (1)
 mvc (1)
 obfuscation (1)
 passenger (1)
 physionet (1)
 praat (1)
 python (1)
 qmake (1)
 rails (1)
 rattle (1)
 regex (1)
 reverse engineering (1)
 sentiment analysis (1)
 signal analysis (1)
 sinatra (1)
 sip (1)
 sqlite (1)
 tika (1)

BLOG ARCHIVE

- ▶ 2018 (2)
 - ▶ 2016 (6)
 - ▶ 2015 (4)
 - ▼ 2014 (6)
 - ▶ November (1)
 - ▼ March (1)
 - Custom tick labels
in R perspective
plots
 - ▶ February (4)
 - ▶ 2013 (8)
 - ▶ 2012 (9)
 - ▶ 2011 (26)
 - ▶ 2010 (8)
 - ▶ 2009 (16)
 - ▶ 2008 (13)
 - ▶ 2007 (10)
-

coordinates of a line at each axis, as described previously. The translated coordinates can be passed directly to `lines()`.

```
lines(trans3d(x.axis, min.y, min.z, pmat) , col="black")
lines(trans3d(max.x, y.axis, min.z, pmat) , col="black")
lines(trans3d(min.x, min.y, z.axis, pmat) , col="black")
```

Drawing Tick Marks

Adding tick marks requires calculating the position of a *second* line, parallel to the axis, and using `segments()` to draw ticks that span the distance between the axis and the second line. The basic procedure is as follows:

```
tick.start <- trans3d(x.axis, min.y, min.z, pmat)
tick.end <- trans3d(x.axis, (min.y - 0.20), min.z, pmat)
segments(tick.start$x, tick.start$y, tick.end$x, tick.end$y)
```

Note the `(min.y - 0.20)` in the calculation of `tick.end`. This places the second line, parallel to the X axis, at the position -0.20 on the Y axis (i.e., into negative/unplotted space).

The tick marks on the Y and Z axes can be handled similarly:

```
tick.start <- trans3d(max.x, y.axis, min.z, pmat)
tick.end <- trans3d(max.x + 0.20, y.axis, min.z, pmat)
segments(tick.start$x, tick.start$y, tick.end$x, tick.end$y)

tick.start <- trans3d(min.x, min.y, z.axis, pmat)
tick.end <- trans3d(min.x, (min.y - 0.20), z.axis, pmat)
segments(tick.start$x, tick.start$y, tick.end$x, tick.end$y)
```

Adding Tick Mark Labels

The final step is to label the ticks on each axis. Once again, the procedure is to calculate the position of a line, parallel to the axis, at the position where the labels are to be displayed:

```
labels <- c('first', 'second', 'third', 'fourth', 'fifth',
'sixth', 'seventh', 'eighth', 'ninth', 'tenth')
label.pos <- trans3d(x.axis, (min.y - 0.25), min.z, pmat)
text(label.pos$x, label.pos$y, labels=labels, adj=c(0, NA),
srt=270, cex=0.5)
```

The `adj=c(0, NA)` expression is used to left-justify the labels, the `srt=270` expression is used to rotate the labels 270°, and the `cex=0.5` expression is used to scale the label text to 75% of its original size.

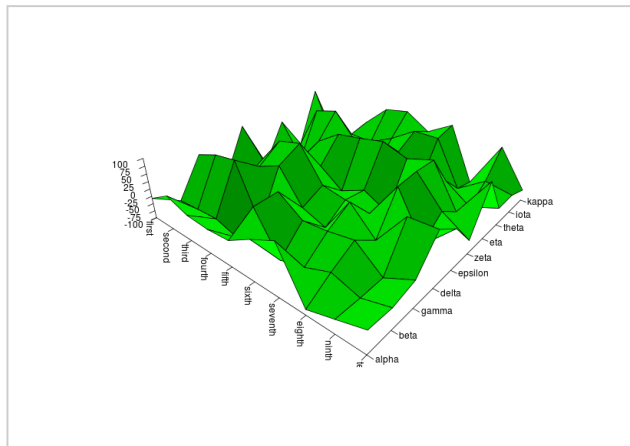
The labels on the Y and Z axes are produced similarly:

```
labels <- c('alpha', 'beta', 'gamma', 'delta', 'epsilon', 'zeta',
'eta', 'theta', 'iota', 'kappa')
label.pos <- trans3d((max.x + 0.25), y.axis, min.z, pmat)
text(label.pos$x, label.pos$y, labels=labels, adj=c(0, NA),
cex=0.5)

labels <- as.character(z.axis)
label.pos <- trans3d(min.x, (min.y - 0.5), z.axis, pmat)
text(label.pos$x, label.pos$y, labels=labels, adj=c(1, NA),
cex=0.5)
```

Note that the Y and Z axis tick labels do not need to be rotated.

The Final Product



POSTED BY MKFS AT 12:56 AM
LABELS: R

15 COMMENTS:



Unknown June 2, 2014 at 1:09 PM

Hello,

Thank you for this code, I found it extremely helpful. I spent hours trying to figure out how to add annotation to axis ticks.

I wonder if we can package in a nice way for better flexibility!!

Reply



Unknown June 2, 2014 at 1:11 PM

Also, in the pmat function, you used , and "" just after persp, a typo!!

Reply

Replies



mkfs June 8, 2014 at 8:21 PM

Yeah, took me ages to figure out so I thought it worth documenting.

Thanks for the typo -- blogger tries to fix source code for you, inevitably breaking stuff. Hopefully it's fixed now.

Reply

Anonymous January 17, 2015 at 6:08 PM

Hi !

Just a message to thank you a lot for your post !
It saved me because i was looking for some code to print persp plots with no values on the z-axis.
I used trans3D as you described to get draw something like i want from a persp graph with no box and no axes;
I spent hours to manage this !!

P.S, forgive my english, i'm french so my sentences could seem strange :p

Reply



Prem May 27, 2016 at 6:24 AM

can you give example with rsm

Reply

Anonymous February 7, 2017 at 2:49 AM

reading this in 2017. thank you so much!

Reply



Unknown January 22, 2018 at 2:04 PM

Excellent example, thanks! Could you please tell me how to have the axes labels after these actions?

Reply



Unknown January 22, 2018 at 2:05 PM

Excellent example! any idea on how to add x.lab, y.lab, z.lab after these actions?

Reply

Replies



mkfs January 26, 2018 at 2:46 PM

persp() accepts the arguments xlab="X axis label", ylab="Y axis label", zlab="Z axis label."

**Unknown** March 11, 2018 at 3:48 PM

Hi mkfs. Thank. you. so. much! I have been trying to work this out for much longer then i care to admit and your directions are perfect!

I am finding that although persp does accept xlab and ylab, it seems to be overridden (ignored) with box=FALSE and axes= FALSE. Any ideas what i am doing wrong? Mostly, thank you!

**Unknown** March 11, 2018 at 4:02 PM

Wait! i think i worked it out!! (by simply making another text object and positioning it in relevance to the axis parameters we created before Horay!!

```
x.axisLabel<-32
xLabel.pos <- trans3d(x.axisLabel, (min.y - 4),
min.z, Plot3DAc)
text(xLabel.pos$x, xLabel.pos$y, labels=xLabel,
srt=320, adj=c(1, NA), cex=0.8)
```

**mkfs** March 18, 2018 at 6:46 PM

Ah, OK - persp probably uses axis or box to determine whether (and where) to display axis labels, and if they are missing/disabled, decides that axis labels should be nixed too.

That trans3d+text fix that you found is the reliable way to add anything to a 3-D plot. Text, points, lines ... use trans3d to determine where they should go in the plot, and everything will work out OK.

I've soured on 3-D plots, though people seem to like them. With complex surfaces, data gets obscured if the plot is fixed (e.g. printed or included in a PDF). I've been using Matplotlib (check out RPyPlot @ <https://github.com/mpastell/Rpyplot>) to pair a 3-D plot with a contour plot, and it seems to help.

Reply**Unknown** September 21, 2018 at 8:51AM

Thank you so much for the example. So useful. I wonder how can I reverse the z-axis values, in such a way the z maximal value is in the lower part of the axis. Thanks.

Reply**Replies****mkfs** October 18, 2018 at 1:08 PM

Since the Z axis has to be constructed anyways, it's probably simplest to negate all the values of the Z axis (e.g. do an `sapply(..., function(x) x * -1)`), and use custom tick labels with the original values - or a tick label format function that multiplies the value by -1 to obtain the original value.

If you try to reverse the direction of the original

values, you're likely going to be fighting the plot function the whole way.

Reply



Unknown July 25, 2020 at 9:29 AM

Please tell how Greek letter write in axis of 3D plot

Reply



Enter Comment

[Newer Post](#)

[Home](#)

[Older Post](#)

[Subscribe to: Post Comments \(Atom\)](#)